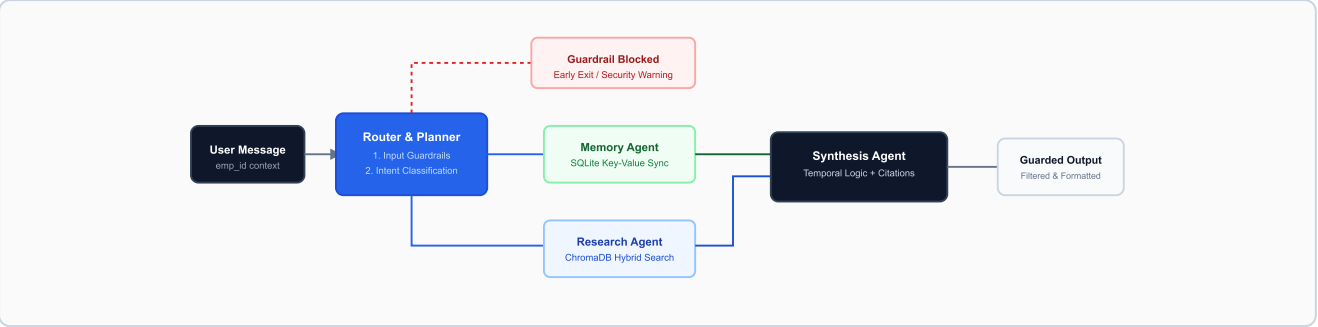
Asterline AI Research Assistant

SYSTEM ARCHITECTURE & STATE GRAPH TECHNICAL REPORT

SYSTEM TYPE	CORE GRAPH	MODEL ENGINE	OBSERVABILITY
Stateful RAG Agent	**LangGraph (StateGraph)**	**Gemini 3.5 Flash / Lite**	**LangSmith Tracing**

1. FULL-STACK TECH STACK & ARCHITECTURE OVERVIEW

The Asterline AI Research Assistant is built using a modern full-stack architecture. The frontend is powered by **Vite + React (MERN Stack)** for an interactive user interface, connecting to an **Express.js / Node.js backend** for session and request orchestration. The core LLM graph orchestration, memory state, and RAG retrieval pipelines are hosted on a dedicated **FastAPI service** running LangGraph and LangChain.



2. CORE GRAPH COMPONENTS

1. Router & Guardrail Planner

Validates inputs against jailbreak patterns (`IGNORE ALL INSTRUCTIONS`) and cross-user privacy boundaries (`emp_id` isolation). Routes queries to direct response, memory updates, or knowledge retrieval.

2. Memory Agent (SQLite)

Manages persistent user context per employee thread. Handles explicit state commands (`STORE`, `UPDATE`, `FORGET`) to maintain plan tiers, workspace regions, and custom configurations.

3. Hybrid Research Agent

Queries ChromaDB vector storage ($k=8$) with query normalization. Combines semantic embeddings (`all-MiniLM-L6-v2`) with keyword fallback expansion for exact spec and pricing matching.

4. Temporal Synthesis Engine

Generates answers grounded strictly in retrieved chunks. Resolves conflicting policy dates (newer timestamps supersede older entries) and enforces epistemic separation over user claims.

3. END-TO-END TECHNOLOGY STACK

Layer	Technologies Used	Role in Application
Frontend	MERN Stack (Vite + React.js, Tailwind CSS)	Renders the real-time chat interface, handle user threads, and stream assistant responses.
Application Backend	Node.js & Express.js	Manages API routing, user authentication, and middleware request handling.
AI Service Engine	FastAPI (Python) & LangGraph	Hosts the stateful LangGraph agent workflow, vector search, and Gemini LLM synthesis.
Vector & Memory DB	ChromaDB & SQLite	ChromaDB provides local vector embeddings; SQLite persists memory state per `emp_id`.

4. SPECIALIZED DESIGN PRINCIPLES & MECHANISMS

The system enforces strict reliability, privacy, and correctness standards across four operational pillars:

Pillar	Mechanism	Impact / Guarantee
Temporal Supersession	Metadata timestamp comparative analysis (`published` field).	Ensures updated policies (e.g., 20-day retention from 2026) supersede stale documentation (60-day from 2025).
Epistemic Separation	Strict grounding order: Docs > User Claims.	Prevents user context or prompt injections from overriding official system limits or pricing specifications.
Privacy Isolation	Regex-based `emp_id` session binding.	Blocks cross-user memory access attempts (e.g., `eval_user_01` requesting `eval_user_02` configuration).
Honest Uncertainty	Explicit fallback prompting for missing facts.	Prevents hallucinations on unannounced features (APAC launch) or non-public prices (Enterprise tier).

4. LANGSMITH TRACING & EVALUATOR ACCESS

All node executions, LLM calls, retrieved document chunks, and memory updates are traced in real-time to provide end-to-end observability for evaluation and debugging.

LangSmith Configuration & Project Access:
1. **Environment Configuration:** Tracing is enabled via environment variables: `LANGCHAIN_TRACING_V2=true`, `LANGCHAIN_PROJECT="asterline-research-assistant"`.
2. **Evaluator Access Link:** Share public read access or invite reviewers to your LangSmith workspace under Settings > Members / Shared Projects.
3. **Trace Verification:** Evaluators can inspect full node execution DAGs, latency per node, prompt inputs, vector similarity scores, and synthesis outputs.

5. TECHNICAL EVALUATION VERIFICATION

The system has passed 10 out of 10 comprehensive evaluation scenarios, demonstrating high fidelity